

Parallel Insertion Merge

Fernando Belmiro do Couto¹ and Fabio Silva do Couto²

¹Divisao de Ti, BBDTV, Rio de Janeiro, RJ, Brasil

²UFRJ, Rio de Janeiro, RJ, Brasil

PDPTA

Abstract – Parallel Insertion Merge is an algorithm that maps the subsets, ordering them with merge and insertion routines and using parallel processing. Most sort algorithms in use nowadays treat the mass of data without previously analyzing its distribution, no matter if dealing with partially ordered datasets. According to the methodology applied in this algorithm the whole data is in the worst case ordered each two elements. According to the same method we prove that at random distribution, are statistically distributed among subsets of 2 + 3 elements. The major parallel routine order two subsets using X threads in the lowest subset, dividing this subset in X parts, and searching the position of the edge elements in highest subset. The next step starts without consider the edge elements used and if they point to the same target, not using the elements between them, because all those elements have their positions assigned.

1. Introduction

Merge is the fastest way to group ordered lists of data, and binary search is the fastest way to seek one element position in a list. We can increase the speed using multithreads and if we have large ordered lists.

Merge, insertion, multithreads and ordered lists are the focus of this algorithm. This is not a insertion sort algorithm variant, since insertion sort use one element each time and this algorithm often insert blocks of elements, taking advantage of the data distribution. *This algorithm tends to use the multiprocessing capability of current computers and is able to adapt itself to increasingly coprocessors quantity. The algorithm first scans a sequence of N elements, comparing each element with the next, verifying if they are ordered according to a previously established criterion and gathering them on positive or negative value subsets if they obey or not that criterion respectively.*

There is order in chaos it depends the way we look, in the first task of the algorithm we will search for order.

2. Mapping Data

The algorithm first scans a sequence of N elements, comparing each element with the next, verifying if they are ordered according to a previously established criterion and gathering them on positive or negative value groups if they obey or not that criterion respectively. At the end of this process, if the distribution is ordered, we have only one index which get a positive N value. On the other hand, if the

distribution is inversely ordered, this index will get negative value (Fig. 1) Assuming other hypothesis, index will range from 2 to $(N / 2) + 1$ in proportion to total data. On average we will have $2N/5$ index. To improve speed we divide the mass of data into fixed sized pieces and implement this in several threads adding the last index to the next index in the next piece according to the value. The last element of a piece is the first element of the next piece.

2	4	6	3	1	9	8	5	7	10	SEQ1
3			-2		-3			2		Index
1	2	3	4	5	6	7	8	9	10	SEQ1
10									Index	
10	9	8	7	6	5	4	3	2	1	SEQ1
-10									Index	

Figure 1.

3. Worst Case

For any distribution the worst case for implementation of the algorithm to initial 4 elements is described as shown in Table I:

TABLE I.

INITIAL 4 ELEMENTS {A,B,C, D...}				
A < B	B < C	C < D	Index(0)	Index(1)
TRUE	FALSE	TRUE	2	2
TRUE	FALSE	FALSE	2	-2
FALSE	TRUE	FALSE	-2	-2
FALSE	TRUE	TRUE	-2	2

The worst case is the way mergesort begins sort.

4. Average Case

As we see, the worst case splits the distribution into two ordered elements groups. We can then infer that on a random distribution the probability to occur 3 ordered elements is 50%. Therefore a minimal elements group to define a random distribution would be 2+3 elements type. This feature is advantageous to mass ordering against Merge

Sort algorithm, which in the beginning divide the distribution at each two elements, sort then and merge the resulting groups until the end. We realize 100,000 Monte Carlo Simulations using 100,000 randomized integers and always obtaining 41.318 %, near 40% expected for phenomenon $2 + 3$.

5. Optimal Order

Grouping data according to the smallest group size optimizes non parallel merge routines performance. Nevertheless with multiprocessing the major factor to time optimization is the number of threads on simultaneously work. This algorithm always uses the lowest subset to choose elements that will seek their position in the greatest subset and gathering their own final position. When two or more elements point to the same insertion place the elements between then have their final positions assigned too.

6. Parallel Insertion Merge

The major algorithm routine order two subsets using X threads in the lowest subset (B), dividing this subset in X parts, and searching the position of the edge elements in the greatest subset (A) using Binary Search. At each step the edge elements of subset B have their position assigned and the block of elements between them when two or more elements have the same target (Fig. 2). When the first element of subset B is greater than some elements of subset A, those elements of A have their position assigned (Fig. 3). In the other hand when the last element of B is less than some elements of A those elements are positioned too (Fig. 4). As we can see in some situations we don't need to compare the elements between threads pointers with same target or when outers pointers having targets inside the greater subset.

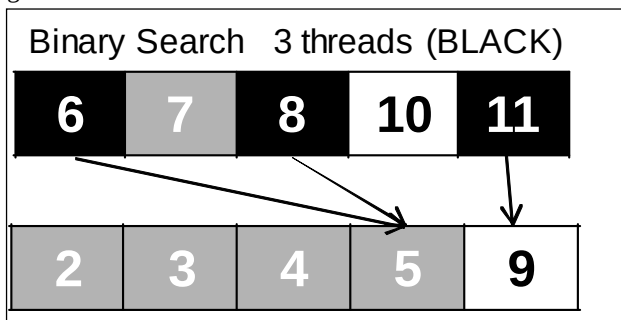


Figure 2

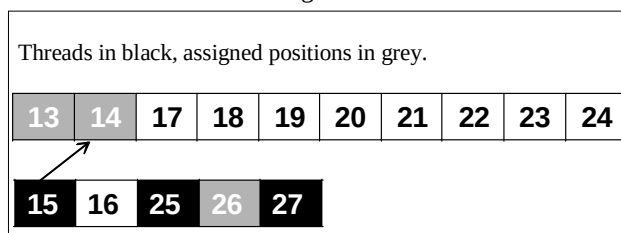


Figure 3

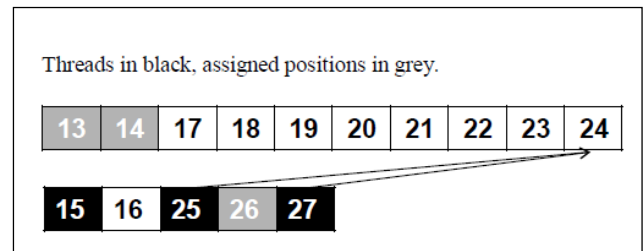


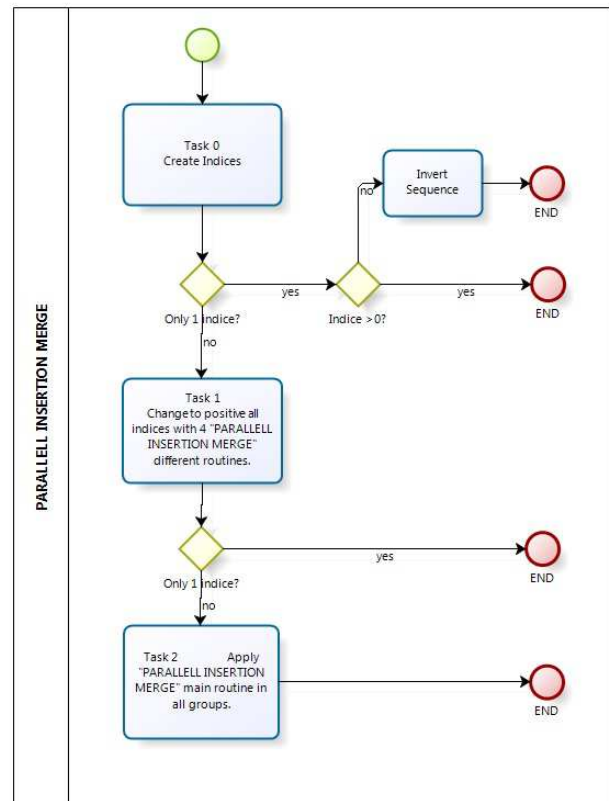
Figure 4

7. Data Conflicts

We have no problems with data conflicts because threads have access to the same element in read routines only, and when threads are writing they set the element in its final position.

8. The Algorithm

The algorithm is composed basically by 3 major Tasks:



The function of Task0 is to identify in the sequence, ordered and inversely ordered groups. To improve this task the mass of data is divided into groups of $N/m + 1$ elements and parallel threads identify indexes. If they are ordered or inversely ordered the next task are not started and the process is redirected to a conclusion.

If the sequence is not ordered Task1 starts. The function of Task1 is to get each group of subsets and call one of 4 recursive routines to merge according ordered subsets or

inversely ordered subsets into one list. This routine takes the initial mapping to optimize the sorting, because if the sign of the index of adjacent groups are opposing, a position already be set (Table II).

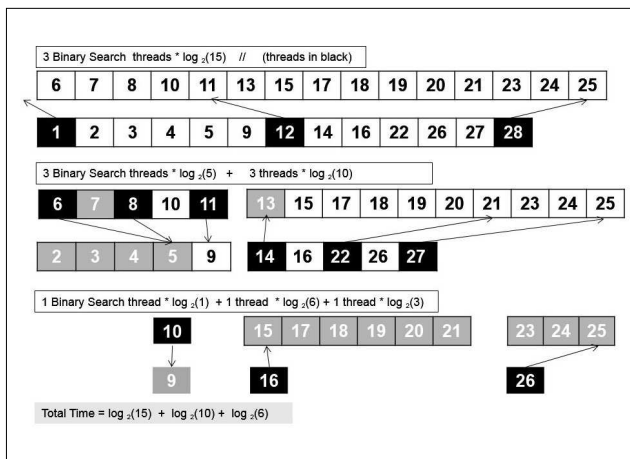
4 ELEMENTS {...,A,B,C, D...}					
$A < B$	$B < C$	$C < D$	Index (6)	Index(7)	Position
TRUE	FALSE	TRUE	2	2	
TRUE	FALSE	FALSE	2	-2	B greater
FALSE	TRUE	FALSE	-2	-2	
FALSE	TRUE	TRUE	-2	2	B minor

Table II

If we have more than 1 indices Task 2 starts. The function of Task 2 is to apply Insertion Merge in each remaining group.

The algorithm terminates when Task2 have only one index and its value is equal to N.

The definition of the maximum number of simultaneously threads depending on the characteristics of the hardware. In this paper we choose 3 to explain, but hundreds of threads may be used to do the job.



9. Conclusion

The purpose of this algorithm is:

- identify pre-existing organizations in the mass of data;
- seek the smallest number of iterations for the mass of data;
- distribute the problem in order to fully utilize the processing hardware capabilities;
- consume the smallest possible space allocation, to achieve the solution of the problem in less time.

I hope this text will contribute to the improvement of the processes of sorting (using insertion and merging) and help fellow developers around the world.

10. Acknowledgments

To our families.

11. References

- [1] CORMEN, Thomas H., Introduction to Algorithms, Second Edition, McGraw-Hill Book Company 2001.
- [2] MIT 6.172 Lecture 13 Analysis of multithreaded algorithms http://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-172-performance-engineering-of-software-systems-fall-2009/lecture-notes/MIT6_172F09 lec13.pdf
- [3] SZWARCFITER, Jaime Luiz; MARKENZONI, Lilian, (1994) Estrutura de dados e seus algoritmos, Editora LTC 2ª Edição.
- [4] BUCKNALL, Julian; Algoritmos e estruturas de dados com Delphi, Editora Berkeley.
- [5] CUDA, GPU COMPUTING, <https://developer.nvidia.com/resources>